Newton School of Technology

Deep Learning — Lecture 1

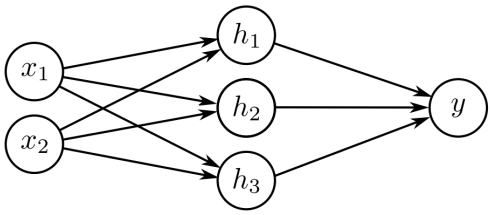
From the Perceptron to a Labelled Shallow Network

Lecture Credo

> *"What I cannot create, I do not understand."*
> — **Richard Feynman**

In this worksheet we will learn about:

- A Brief History of AI with Deep Learning
- Why a single neuron isn't enough — XOR and the case for hidden layers
- The Shallow Neural Network Architecture & Notation
- The perceptron as a linear decision rule
- Why XOR defeats a single linear boundary
- How a hidden layer repairs the XOR limitation
- Input, hidden, and output layer roles
- Forward-propagation notation and tensor shapes



Student Name: _____

Student ID: _____

Semester: _____

CLOs: explain why a single perceptron fails on non-linearly-separable data; show how a hidden layer resolves XOR; construct and label a shallow network; and trace the shapes and notation used in one forward pass.

Primary Reference: S. J.D. Prince, *Understanding Deep Learning*, MIT Press (2023), Chapter 3 — Shallow Neural Networks.

How to Use This Worksheet

This worksheet is built to be used **during** class, not after it.

1. Your instructor teaches a chunk of slides (usually 10–12).
2. You come to the matching **teal Recap box** — labelled with the exact **lecture and slide range** it covers, e.g. *"Lecture 1 — Slides 3–12."* Read the recap.
3. You attempt the **Quick Check** below it, *without* looking back at the slides.
4. Your instructor (or you, self-checking) can now see whether the concept landed, before moving on.

Stuck on a Quick Check? Re-read the recap box right above it, not the full slide deck.

Lecture 1 — From Perceptrons to a Labelled Architecture

Setting the Context, the MLP Architecture, and Forward-Propagation Notation.

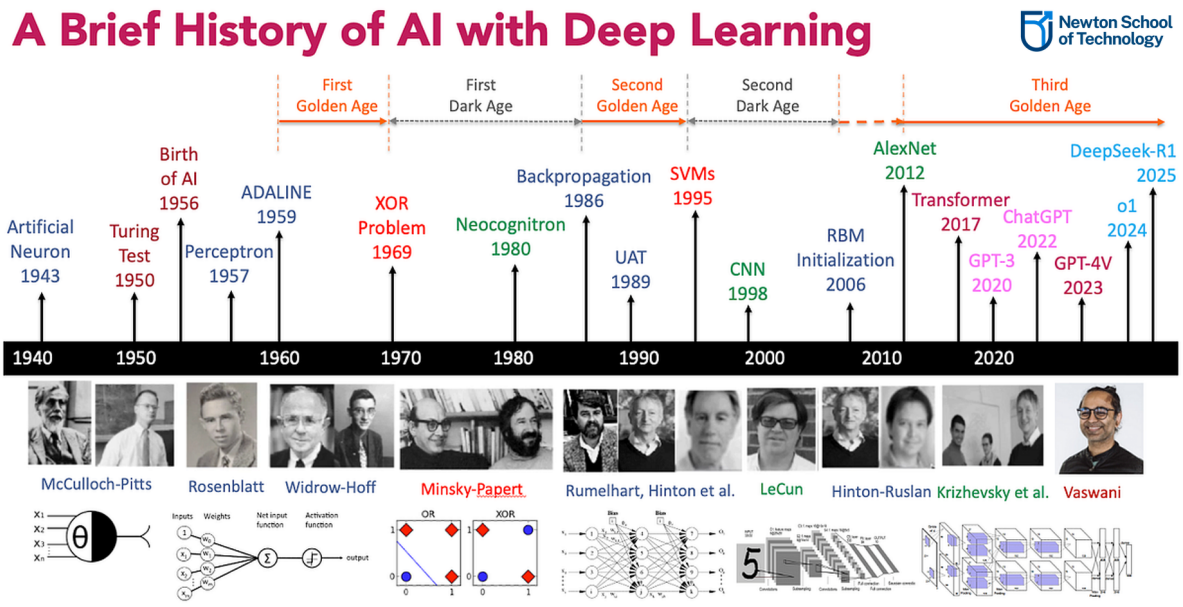
1 Setting the Context — Why Neural Networks?

We begin not with a definition, but with a question. The goal of this section is to make you *feel the need* for a hidden layer before we formally introduce one. But first, a map of where today's lecture sits in the larger story of AI.

A Brief History of AI with Deep Learning

Recap — Lecture 1, Slide 6: Three Golden Ages, Two Dark Ages

AI's history is not one smooth climb — it moves in **golden ages** and **dark ages** ("AI winters"). The **First Golden Age** opens with the Turing Test (1950) and the "Birth of AI" at the 1956 Dartmouth conference, followed by the Perceptron (Rosenblatt, 1957) and ADALINE (Widrow–Hoff, 1959) — the first learning machines. It ends abruptly in 1969, when Minsky and Papert proved a single perceptron cannot solve XOR, and funding collapsed into the **First Dark Age**. The **Second Golden Age** begins in 1986, when Rumelhart, Hinton, and Williams popularized **backpropagation** — a way to train networks *with* hidden layers, directly solving the XOR problem that started the winter; the Universal Approximation Theorem (1989) then gave that revival a theoretical foundation. But by 1995, Support Vector Machines out-performed neural networks on most practical tasks with far less computation, opening a **Second Dark Age** for neural nets (even as CNNs, LeCun 1998, quietly kept the idea alive). The **Third Golden Age** — the one we are living in now — is dated from Hinton's RBM pretraining breakthrough (2006) and confirmed by AlexNet's 2012 ImageNet win. The Transformer (2017) then triggered the large-language-model race: GPT-3, ChatGPT, GPT-4V, and today's reasoning models.



A Brief History of AI with Deep Learning — from the McCulloch–Pitts artificial neuron (1943) to

DeepSeek-R1 (2025).

Read the Timeline

Using the timeline above, answer the following.

1. Fill in the year for each milestone: Perceptron (_____), Backpropagation (_____), AlexNet (_____), Transformer (_____).
2. Which **Age** does each of these fall into — write First/Second/Third Golden Age or First/Second Dark Age:
   - ADALINE (1959): _____
   - SVMs (1995): _____
   - ChatGPT (2022): _____
3. You are about to spend the next hour learning to solve the XOR problem by hand, using hidden layers. Looking at the timeline, roughly how many years passed between XOR freezing the field (1969) and backpropagation — the fix you are about to learn — reviving it (1986)? _____
4. Every "Dark Age" on this timeline was ended by the *same* kind of event. In one sentence, what pattern do you notice? _____

Recap — Lecture 1, Slides 3–12: The Perceptron & the Limits of a Straight Line

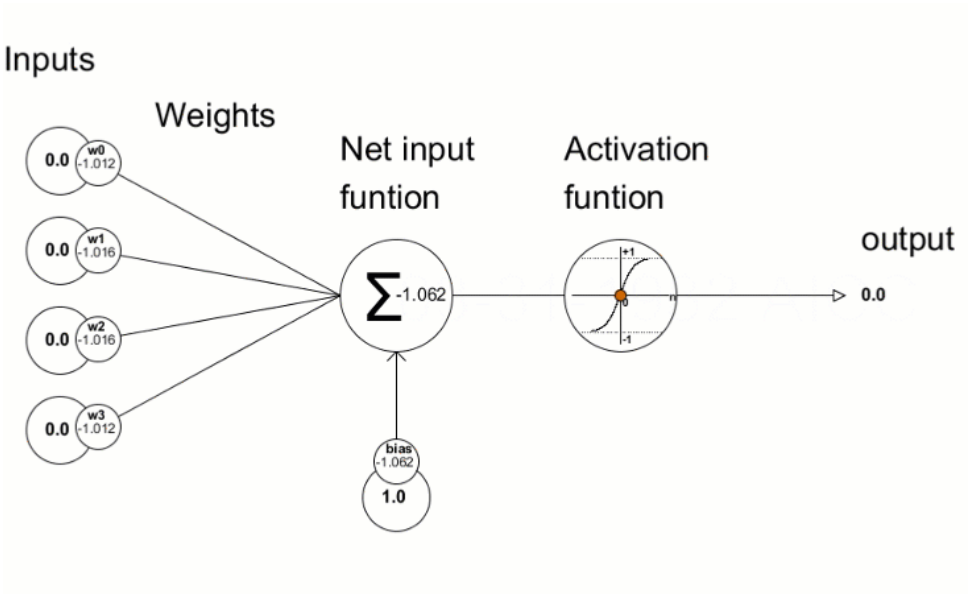
Rosenblatt (1958) introduced the **perceptron**: it linearly combines its inputs and then thresholds the result to make a yes/no decision, $y = \phi\left(\sum_i w_i x_i + b\right)$. Geometrically, $\sum_i w_i x_i + b = 0$ is a hyperplane (a straight line in 2D) that cuts the input space in two; a single perceptron can only ever place its decision boundary along *one* such straight cut. This works whenever the two classes are **linearly separable** — exactly the case for AND and OR below, where one straight line cleanly separates the 1s from the 0s. But Minsky and Papert (1969) proved that a single perceptron cannot represent every possible mapping: some datasets have no straight-line boundary at all. The Quick Checks below walk you through exactly that limit.
Now, using this idea, attempt the checks below.

Terminology Alert — The Perceptron

A perceptron computes a weighted sum of its inputs and passes the result through an activation function to produce a single output:

$$y = \phi\left(\sum_i w_i x_i + b\right)$$

where x_i is the i-th input feature, w_i its weight, b the bias term, and ϕ the activation function. This is structurally identical to **logistic regression** when ϕ is the Sigmoid function — a perceptron is a single logistic regression unit.



A perceptron with 4 inputs: each input is multiplied by its weight, summed with the bias, and passed through an activation function to produce the output.

Match the Symbol

In the perceptron equation from the recap, $y = \phi\left(\sum_i w_i x_i + b\right)$, match each symbol to what it represents. Write the correct letter next to each symbol.

Symbol	Meaning
x_i	____
w_i	____
b	____
ϕ	____

(A) The activation function
(B) The bias term
(C) The i-th input feature
(D) The weight on input i

Building Intuition: The Limit of Linearity

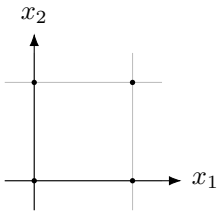
Terminology Alert — Linearly Separable

A dataset is **linearly separable** if a single straight line (in 2D) — or a hyperplane in higher dimensions — can perfectly separate its classes.

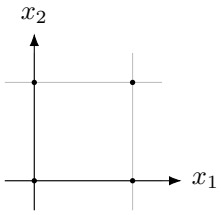
Fill In the Truth Tables

AND and OR are both linearly separable — a single straight cutoff can separate them. Fill in the AND and OR truth tables below, then mark each point on the grid next to it (**draw a circle around** the dot for output 1, **draw an X through** the dot for output 0 — two different shapes so the distinction survives photocopying), and try sketching one straight line that separates the circled points from the crossed-out ones.

x_1	x_2	AND
0	0	____
0	1	____
1	0	____
1	1	____



x_1	x_2	OR
0	0	____
0	1	____
1	0	____
1	1	____



Could you draw one straight line that separates the circled points from the crossed-out ones, for both AND and OR? ☐ Yes ☐ No

So far, so good: AND and OR are both linearly separable, so a single perceptron handles them fine. But linear separability is a *constraint*, not a guarantee — it does not hold for every possible mapping. Turn the page and try the same one-line trick on XOR.

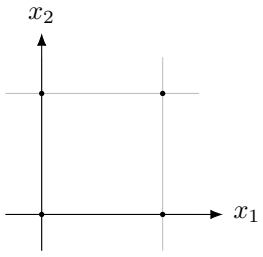
Recap — Lecture 1, Slides 13–16: The XOR Wall — and the Fix

Picture a staircase light controlled by two switches, one at the top and one at the bottom — a wiring pattern used in real houses. Flipping *either one* switch turns the light on. Flipping *both* switches (from off) turns it back off, because the wiring cancels out. That behaviour — "on when exactly one switch differs from the other, off otherwise" — is precisely XOR ("exclusive or": one or the other, but not both). No single straight cutoff line can capture this rule, because the two "light-on" states sit on opposite corners of the truth table from each other. Real electricians solve this with a clever wiring trick that combines two simpler switches; neural networks solve it the same way — with a **hidden layer** that first computes two simpler sub-decisions, then combines them.

Fill In and Plot XOR

Fill in the truth table for the staircase-light rule (XOR), then mark each point on the grid (circle it for output 1, cross it out for output 0), the same convention as before.

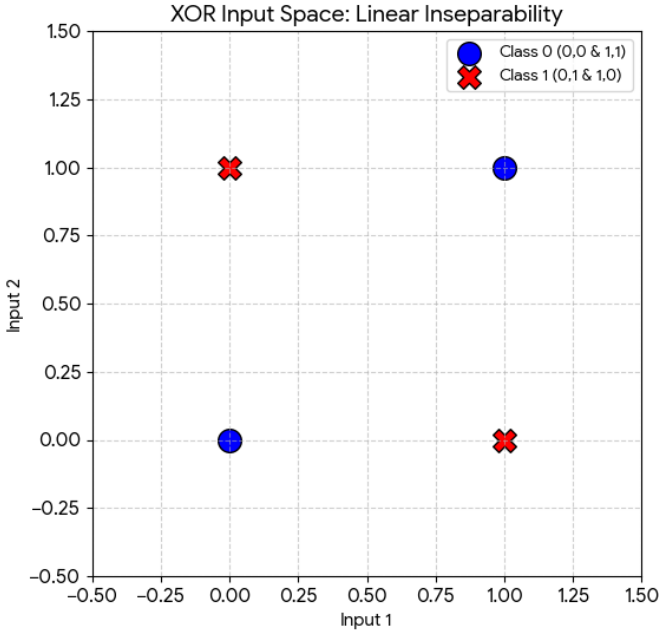
x_1	x_2	XOR
0	0	____
0	1	____
1	0	____
1	1	____



Now try it. Using a pen, attempt to draw *one* straight line on the grid above that puts all the circled points on one side and all the crossed-out points on the other. Try a few different lines — notice that none of them work.

Think Box

What went wrong? Why does a single straight line work for AND and OR but not for XOR? Write your explanation in your own words below.



The XOR classes plotted in input space — no single straight line separates the crosses from the circles.

If you found that you needed *two* lines, or a curved boundary, you have just rediscovered — by hand — the exact limitation Minsky and Papert (1969) proved: a single perceptron draws exactly one straight line, and XOR needs more than that. This result is what triggered the **First AI Winter** on the history timeline at the start of this section — funding and research on neural networks collapsed for over a decade, until backpropagation (1986) provided a way to train the hidden-layer fix you are about to build. To separate XOR, we need something with more than one line to work with.

The Solution — Hidden Layers

Setting Up the Fix

Just like the staircase-light wiring never asks one switch to decide the whole rule alone, what if, instead of asking one neuron to solve XOR directly, we let two *simpler* neurons each solve an easier sub-problem first, and then combine their two answers?

Suppose our two simpler neurons compute the following:

- $h_1 = x_1$ OR x_2
- $h_2 = x_1$ NAND x_2 (NAND is "NOT AND": 1 unless both inputs are 1)

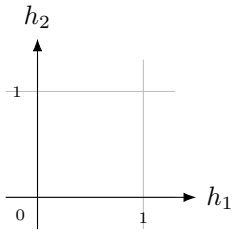
Discover the Trick Yourself

Compute h_1 and h_2 for each row, then compute $y = h_1$ AND h_2 in the last column.

x_1	x_2	$h_1 = x_1$ OR x_2	$h_2 = x_1$ NAND x_2	$y = h_1$ AND h_2
0	0	_____	_____	_____
0	1	_____	_____	_____
1	0	_____	_____	_____
1	1	_____	_____	_____

Compare your final column y to the XOR truth table on the previous page. What do you notice? _____

Now plot (h_1, h_2) for all four rows on the grid below, using the same circle-for-1 / cross-for-0 convention based on the value of y.



Is this new (h_1, h_2) picture linearly separable? ☐ Yes ☐ No

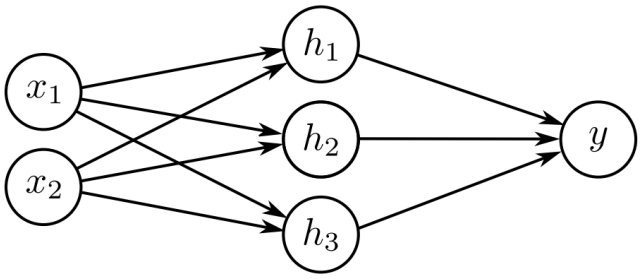
This is the whole trick behind hidden layers: they **transform the input space** into a new representation, where the hidden neurons learn intermediate features (like OR and NAND above). In this new feature space, the data becomes linearly separable for the final output layer to handle. Let's now formalize this into a proper architecture.

2 The Shallow Neural Network Architecture

Recap — Lecture 1, Slides 17–23: Naming the Architecture

Prince's *Understanding Deep Learning* names the three parts plainly. The **input layer** carries the raw features in and does no computation. The **hidden layer** sits between input and output: each hidden unit forms a linear combination of the inputs, passes it through a non-linear activation, and the layer is called "hidden" because its values are never directly observed in the training data. The **output layer** then linearly recombines the hidden units into the final prediction, whose size depends entirely on the problem. A network with exactly one hidden layer is a **shallow neural network**; historically, any network with at least one hidden layer is also called a **multi-layer perceptron (MLP)**. On the last page, one neuron alone could not separate XOR — a hidden layer, with more than one unit, can.

A **shallow neural network** (also called a Multilayer Perceptron, or MLP) has three types of layers: an input layer, one hidden layer, and an output layer.



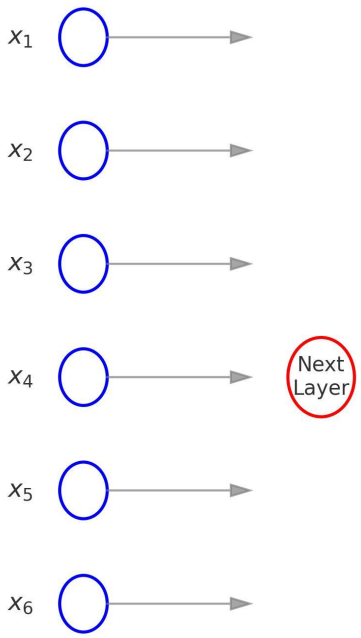
A shallow network: 2 inputs, 3 hidden units, 1 output.

2.1 Input Layer

Terminology Alert — Input Layer

The entry point for raw data into the network. It directly takes in the features of each data sample; in a tabular dataset, each neuron corresponds to one column. No computation happens here.

Input Layer with d Neurons Feeding Next Layer



An input layer with $d = 6$ neurons, one per input feature, each simply passing its value forward to the next layer — no computation happens here.

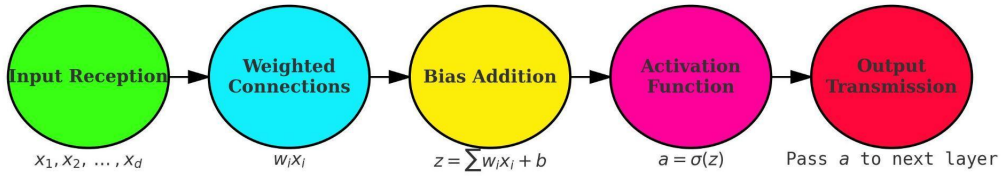
Fill in the blank: the number of neurons in the input layer is determined by the _____ of the input data.

2.2 Hidden Layer(s)

Terminology Alert — Hidden Layer

An intermediate processing layer between the input and output that learns complex, non-linear patterns by transforming inputs through weighted connections and non-linear activation functions. For a *shallow* network specifically, there is exactly **one** hidden layer.

Workflow of a Hidden Layer Neuron



$$a = \sigma\left(\sum_{i=1}^{d} w_i x_i + b\right)$$

where:
- x_i: input features
- w_i: weights
- b: bias
- z: weighted sum ($\sum w_i x_i + b$)
- σ: activation function
- a: output of the neuron

The five-step workflow inside every hidden layer neuron: receive inputs, apply weights, add the bias, pass through the activation function, and transmit the output onward.

Match the Job to the Description

Match each job below to what the hidden layer actually does.

Job	Description
Learning complexity	____
Feature extraction	____
Improved function	____

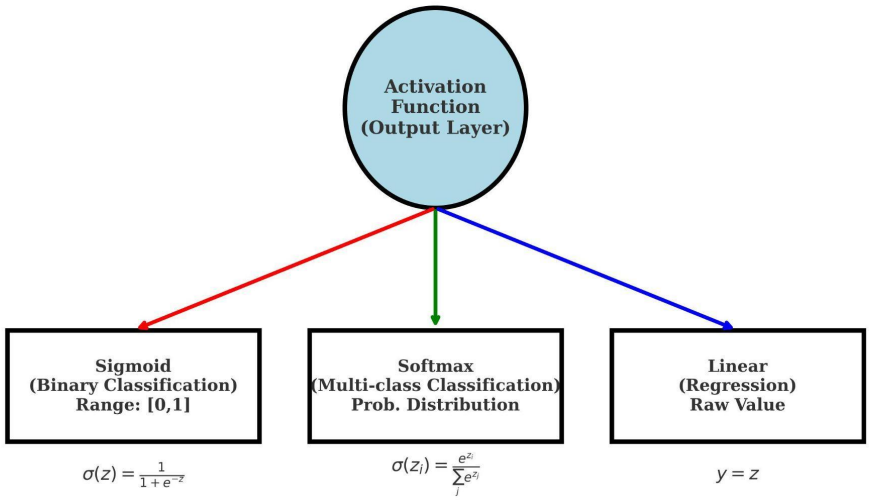
(A) Extracts increasingly intricate features from the raw input
(B) Enables classification/regression a single layer cannot handle
(C) Models and captures non-linear relationships in the data

2.3 Output Layer

Terminology Alert — Output Layer

The final layer that produces the network's prediction or classification. Its size depends entirely on the type of problem being solved.

Choice of Activation Function in Output Layer



The output layer's activation function is chosen by the problem type: Sigmoid for binary classification, Softmax for multi-class classification, and a plain linear unit for regression.

Fill In the Output Size

The number of output neurons depends entirely on the problem type. Fill in the table below.

Problem type	Number of output neurons
Binary classification (spam / not spam)	_____
Multi-class classification with 7 classes	_____
Regression (predicting a house price)	_____

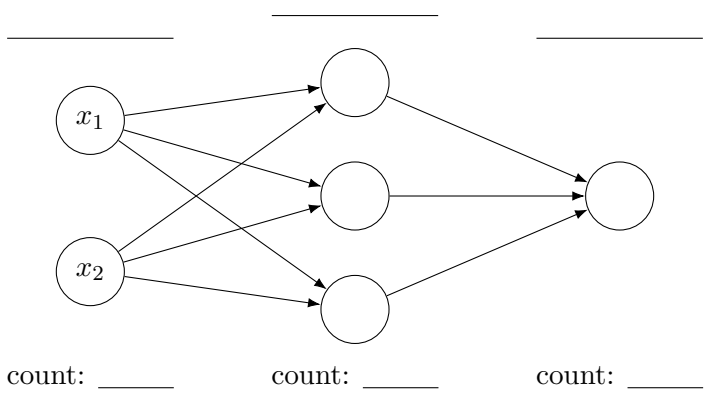
Quick Check

If I have a dataset with 10 input features and I want to classify images into 5 categories:
Input layer neurons = _____ Output layer neurons = _____
Justify your reasoning in one line:

_____

Label the Diagram

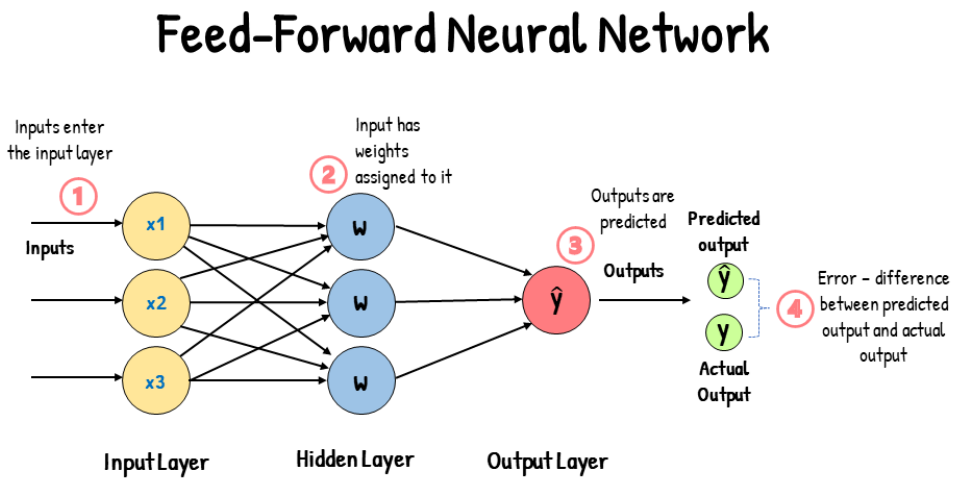
Below is a shallow neural network with 2 input neurons, 3 hidden neurons, and 1 output neuron, fully connected between adjacent layers. **Write "Input Layer", "Hidden Layer", or "Output Layer" in the blank above each column**, and write the neuron count beneath each column.



3 Notation Used in Forward Propagation

Recap — Lecture 1, Slides 17–23: Giving Every Weight an Address

A postal address works because it is layered: country, then city, then street, then house number — each level narrows down exactly one location, with no ambiguity. Weight notation works the same way. $w_{ij}^{[l]}$ is a full address for one connection: $[l]$ is the "city" (which layer the connection lands in), i is the "street" (which neuron in that layer receives it), and j is the "house number" (which neuron in the previous layer sent it). During the forward pass, every layer runs the same two-step delivery: first a linear step $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$ (collect everything addressed to this layer), then a non-linear step $a^{[l]} = \phi(z^{[l]})$ (process it). Try describing "the weight connecting the second hidden neuron to the third output neuron" in plain English instead — for a network with hundreds of neurons per layer, that sentence gets unusable fast. The address notation never does.



The forward pass, step by step: inputs enter, weights are applied, the output is predicted, and compared against the actual output to measure error.

3.1 The Two-Index Weight Notation

Terminology Alert — Weight Notation

Weights are written as $w_{ij}^{[l]}$, where:

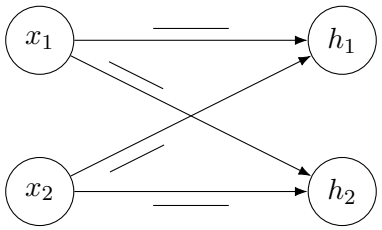
- **Superscript** $[l]$ — the layer the weight belongs to. $W^{[1]}$ is the weight matrix connecting the input layer to the first hidden layer. $W^{[2]}$ connects the hidden layer to the output.

- **Subscript** ij — i is the destination neuron (in the current layer), j is the source neuron (in the previous layer).

Reading w_{11}: it means the weight on the connection going from neuron 1 of the previous layer *to* neuron 1 of the current layer.

Label the Diagram

Below is a small network with 2 inputs and 2 hidden neurons. Four weights connect them. Using the "street/house-number" address rule from the recap, **write the correct label (w_{11}, w_{12}, w_{21}, or w_{22}) on each edge.**



Now you try (fill in the blanks):

w_{32} means the weight from _____

to _____.

$w_{11}^{[2]}$, in a network where layer 2 connects the hidden layer to the output, means:

_____

Matrix Shape

The weight matrix $W^{[1]}$ for a network with 2 inputs and 3 hidden neurons has shape (_____ × _____).

Explain why, using the destination/source idea above:

_____

3.2 The Theta Notation (Some Textbooks)

In some textbooks and slide decks, weights are written as θ instead of w. The logic is identical — only the symbol changes.

Terminology Alert — Reading θ_{i0} and θ_{i1}

Following Prince's *Understanding Deep Learning* notation, each hidden unit i has its own parameters $\theta_{i0}, \theta_{i1}, \theta_{i2}, \ldots$:

- **First subscript** i — which hidden unit this parameter belongs to.
- **Second subscript** — *which input it multiplies*: θ_{i0} is always the **bias** (the 0 marks the "always-on" input, which is not actually multiplied by anything); θ_{i1} multiplies input x_1, θ_{i2} multiplies input x_2, and so on.

So hidden unit i computes $h_i = \mathrm{a}[\theta_{i0} + \theta_{i1}x_1 + \theta_{i2}x_2 + \ldots]$ — the same address logic as $w_{ij}^{[l]}$, minus the layer superscript, since θ is normally used only for single-hidden-layer (shallow) networks.

Match Theta to Its Meaning

Same address logic, different symbol. Assume a single-input network with 2 hidden neurons.

Expression	Meaning
θ_{10}	____
θ_{11}	____
θ_{20}	____
θ_{21}	____

(A) The bias term for hidden neuron 2
(B) The bias term for hidden neuron 1 (the 0 marks the always-on bias input)
(C) The weight from input x to hidden neuron 2
(D) The weight from input x to hidden neuron 1

3.3 Full Notation Summary

Match Symbol to Meaning

These are the "address labels" you will use every single day for the rest of this course.

Symbol	Meaning
x_i	____
$W^{[l]}$	____
$b^{[l]}$	____
$z^{[l]}$	____
$a^{[l]}$	____
ϕ or σ	____

(A) Pre-activation output of layer l, i.e. $z = Wx + b$
(B) Post-activation output of layer l, i.e. $a = \phi(z)$
(C) The activation function, applied element-wise
(D) The weight matrix for layer l
(E) The bias vector for layer l
(F) The i-th input feature

Mathematical Formulation

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = \phi\left(z^{[l]}\right)$$

Python (NumPy)

```
W1 = np.array([[0.2, -0.5],
               [0.1,  0.4],
               [-0.3, 0.7]])
b1 = np.array([0.1, 0.0, -0.2])

z1 = _______ @ x + _______
a1 = phi(z1)
```

The 0-Indexing Gotcha

In our math notation, we count neurons starting from 1 (so w_{21} is a valid weight). In code, array indices start from 0. Which array element, `W1[i][j]`, corresponds to the math weight w_{21} (from input neuron 1 to hidden neuron 2)?
`W1[_____][_____]`

You should now be able to look at *any* weight in a network and say exactly which two neurons it connects, and in which direction. This becomes essential in a later sheet, when we run this whole diagram *backwards* to train the network.

Lecture 1 Exit Ticket

1. Why can one perceptron not solve XOR?

2. Draw a $2-3-1$ network and label $W^{[1]}$, $b^{[1]}$, $z^{[1]}$, $a^{[1]}$, $W^{[2]}$, and $b^{[2]}$.

3. What new capability does the hidden layer introduce?

End of Lecture 1 Worksheet